



Ultimate OOPS Interview Question Sheet (Medium → Hard)

Based on past chats + notes + FAANG-style prep

A consolidated revision sheet for OOPS/Java interviews.

1. OOP Fundamentals

1. What is Object-Oriented Programming?
 2. Why is OOP preferred over procedural programming?
 3. What are the four pillars of OOP?
 4. Is Java purely object-oriented? Why?
 5. Difference between class and object.
 6. What are attributes and methods?
 7. What is message passing in OOP?
 8. What is modularity in OOP?
 9. Real-world examples of OOP.
 10. Why is OOP useful in scalable applications?
-

2. Class, Object & Memory

1. What happens internally when an object is created?
 2. Where are objects stored in Java memory?
 3. Difference between stack and heap.
 4. What is object reference?
 5. Can object exist without reference?
 6. What is garbage collection?
 7. What is object lifecycle?
 8. What is memory leak in Java?
 9. StackOverflowError vs OutOfMemoryError.
 10. Can we create object without new keyword?
-

3. Constructors

1. What is a constructor?
2. Types of constructors.
3. Default vs parameterized constructor.
4. Constructor overloading.

5. Can constructors be inherited?
 6. Can constructors be private?
 7. Can constructors be static?
 8. What is constructor chaining?
 9. Order of constructor execution in inheritance.
 10. What happens if constructor throws exception?
-

4. Encapsulation

1. What is encapsulation?
 2. How does encapsulation provide security?
 3. Why make variables private?
 4. What are getters and setters?
 5. What happens if we don't use getters/setters?
 6. Encapsulation vs data hiding.
 7. Real-world example: banking system.
 8. Can encapsulation be broken using reflection?
 9. Benefits of encapsulation in large systems.
 10. How does encapsulation reduce coupling?
-

5. Inheritance

1. What is inheritance?
 2. Why is inheritance used?
 3. Types of inheritance.
 4. Why Java does not support multiple inheritance with classes?
 5. What is diamond problem?
 6. What is IS-A relationship?
 7. Can private members be inherited?
 8. What is super keyword?
 9. Constructor chaining with super().
 10. When inheritance becomes bad design?
-

6. Polymorphism

1. What is polymorphism?
2. Types of polymorphism.
3. Method overloading vs overriding.
4. Compile-time vs runtime polymorphism.
5. What is dynamic method dispatch?
6. Why variables do not show polymorphism?
7. Can static methods be overridden?
8. Can private methods be overridden?

9. What is covariant return type?
 10. Real-world example of polymorphism.
-

7. Abstraction & Interfaces

1. What is abstraction?
 2. Abstraction vs encapsulation.
 3. What is abstract class?
 4. What is interface?
 5. Abstract class vs interface.
 6. Can abstract class have constructor?
 7. Can interface have constructor?
 8. Default and static methods in interfaces.
 9. What is marker interface?
 10. Why interfaces are used in scalable systems?
-

8. Access Modifiers

1. Explain public, private, protected, default.
 2. What is default access modifier in Java?
 3. Can private method be overridden?
 4. Can constructor be private?
 5. protected access across packages.
 6. Can top-level class be private?
 7. Can child reduce access level while overriding?
 8. Package-private use cases.
 9. Why use least privilege principle?
 10. Access modifiers in API design.
-

9. Static, Final, this, super

1. What is static keyword?
 2. Static variable vs instance variable.
 3. Static method rules.
 4. Why main() is static?
 5. Can static be instantiated?
 6. What is final variable/method/class?
 7. Difference between final, finally, finalize.
 8. What is this keyword?
 9. Difference between this and super.
 10. Why String class is final?
-

10. Association, Aggregation, Composition

1. What is association?
 2. What is aggregation?
 3. What is composition?
 4. Aggregation vs composition.
 5. HAS-A relationship.
 6. Why composition preferred over inheritance?
 7. Real-life examples of composition.
 8. Lifecycle dependency in composition.
 9. Loose coupling using composition.
 10. Refactor inheritance into composition example.
-

11. Object Class & Hashing

1. Difference between == and equals().
 2. Why override equals()?
 3. Why override hashCode()?
 4. equals-hashCode contract.
 5. Can two unequal objects have same hashCode?
 6. How HashMap uses equals and hashCode?
 7. What is toString() method?
 8. clone() shallow vs deep copy.
 9. Identity vs equality.
 10. Common bugs with mutable keys in HashMap.
-

12. String Internals

1. Why String is immutable?
 2. What is String pool?
 3. String literal vs new String().
 4. StringBuilder vs StringBuffer.
 5. Why String is thread-safe?
 6. What happens internally with concatenation?
 7. intern() method use.
 8. Why String used as HashMap key?
 9. Memory impact of strings.
 10. Best practices with strings.
-

13. SOLID Principles

1. What are SOLID principles?

2. Explain SRP with example.
 3. Explain OCP with example.
 4. Explain LSP with example.
 5. Explain ISP with example.
 6. Explain DIP with example.
 7. Real-world SOLID violations.
 8. How SOLID improves maintainability?
 9. SOLID in microservices/backend design.
 10. Which SOLID principle is hardest to apply and why?
-

14. Design Patterns

1. What are design patterns?
 2. Singleton pattern and issues.
 3. Thread-safe Singleton implementation.
 4. Factory pattern use case.
 5. Factory vs Abstract Factory.
 6. Observer pattern example.
 7. Strategy pattern example.
 8. Builder pattern example.
 9. MVC basics.
 10. When should you avoid patterns?
-

15. Exception Handling + OOP

1. What is exception handling?
 2. try-catch-finally flow.
 3. throw vs throws.
 4. Checked vs unchecked exceptions.
 5. Can constructors throw exceptions?
 6. Can finally block not execute?
 7. Can overridden method throw broader exception?
 8. Custom exceptions.
 9. Best practices in enterprise apps.
 10. Designing exception hierarchy.
-

16. Scenario-Based Questions

1. Design ATM system using OOP.
2. Design Library Management System.
3. Design Payment System using interfaces.
4. Design Chat App using OOP.
5. Model Student-Course-Enrollment classes.

6. Car and Engine using composition.
 7. Where would you use inheritance in real project?
 8. How does polymorphism help scalability?
 9. Why use interface instead of concrete class?
 10. Tell a bug caused by bad OOP design.
-

17. MAANG Deep Questions

1. Why composition is preferred over inheritance?
 2. Explain polymorphic dispatch internally.
 3. How JVM resolves overridden methods?
 4. What is method table / vtable concept?
 5. What is early binding vs late binding?
 6. What is object slicing?
 7. What is shadowing in OOP?
 8. What is multiple dispatch?
 9. Why multiple inheritance is risky?
 10. Biggest OOP mistakes that hurt scalability.
-

18. Quick Revision (Must Do)

1. Four pillars of OOP.
2. Class vs Object.
3. Overloading vs Overriding.
4. Abstract class vs Interface.
5. Encapsulation vs Abstraction.
6. Association vs Aggregation vs Composition.
7. this vs super.
8. Static vs Final.
9. equals vs ==.
10. SOLID principles.